



ACTIVITY LOGBOOK

Perabots

2025

Faculty of Engineering,
University of Peradeniya



ACTIVITY LOGBOOK

By Team Robot Wanderers

Members:

Sanindu Hansara

Piyumi Nipunika

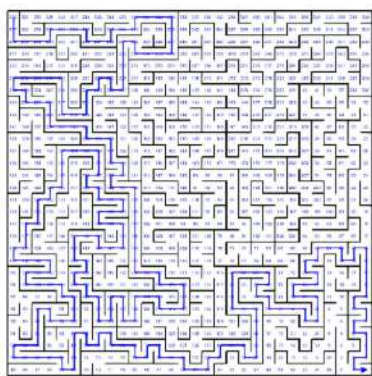
Tharani Pathirana

Thisum Abeywickrama

**FACULTY OF ENGINEERING,
UNIVERSITY OF PERADENIYA**

Before the task releasing date.

We explored the webots software, interface and basic robot control functionalities. We tested a few fundamental algorithms such as wall following. Tried to insert sensors to the epuck and tried some codes which operates sensors. Then we learned how to insert a camera to e-puck. thought the task would be something related maze solving. Therefore we started searching algorithms such as floodfill, A* search and wall following algorithms. We used above three algorithms and tried small maze solving projects. However, finally we came to the decision that floodfill would be more suitable and easy for us to use incase the task was maze solving.



This is how maze solving happens using floodfill algorithm.

References we used to learn the above algorithms.

Date : 09th of May 2025

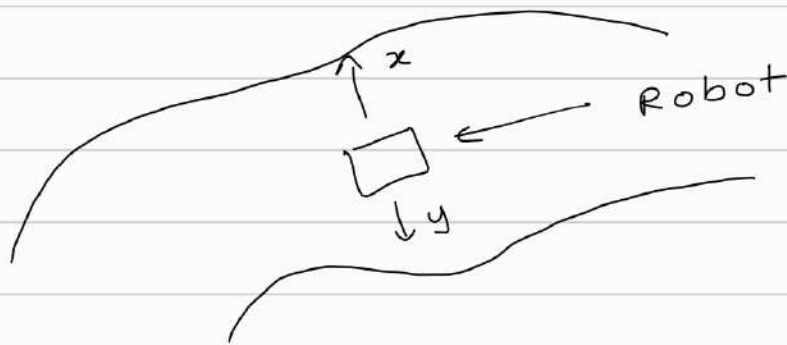
The task was released this week. It was not a maze solving project but an entirely different task. The example arena was released today.

First, we decided to use the IR sensors for our robot, but it started malfunctioning sometimes near the walls. Later we could identify that this is because of the shadows casted from the wall. Therefore we decided to use distance sensors for our robot.

Then we tried to take readings near left and right walls. Even though, the sensors gave a sensible reading near the outer wall, it did not sense the inner wall altogether. Later we understood that something is wrong with the wall and, we decided to fix it.

Date : 10th of May 2025

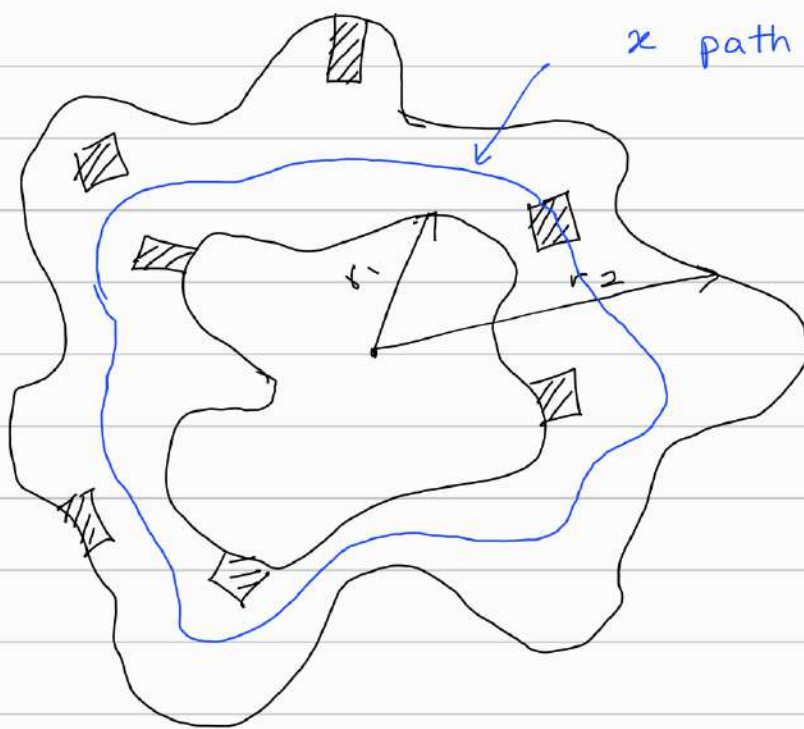
We brainstormed ways to complete the task. Lastly we came up with the following method. Send the robo in the middle of the path by fixing sensors to it and getting x and y values as shown in the following picture. Then send it in a path where $x = y$.



But this was not very successful because the distance sensors could not measure x and y properly. This is because the width of the road was beyond the capacity of the sensor.

Date : 11th of May 2025

We decided to send the robo following the left wall for the first round and the second round we decided to send the robo following the right wall. During these two runs we planned to take the r_1 and r_2 values for every position in the arena and store them in an array. Then in the 3rd round, the robot will be sent in a path marked as x in the following diagram.

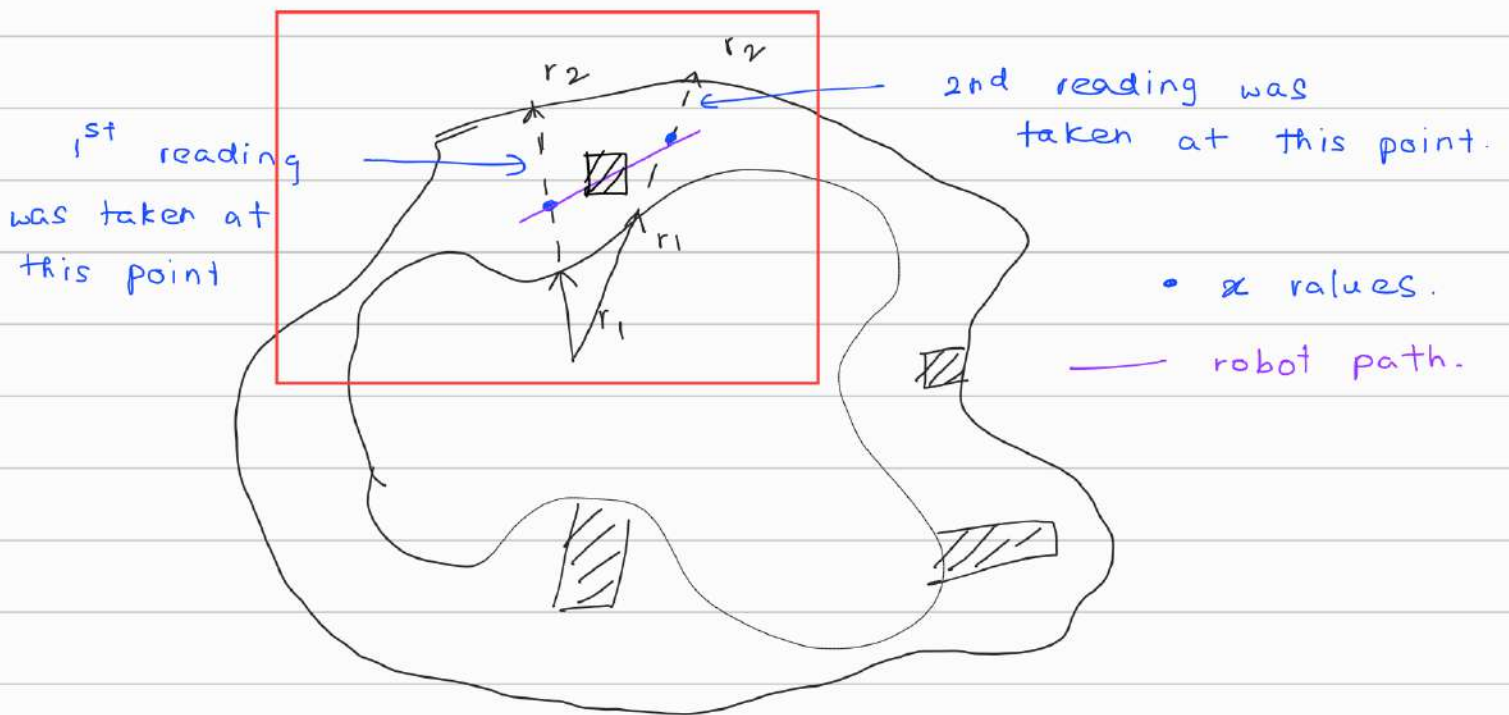


$$x = \frac{r_1 + r_2}{2}$$

Date : 14th May 2025

We tried to implement the method we planned yesterday. But the following problems were encountered

→ The main problem is we cannot take a r_2 value for every r_1 value in the path. This because they have two different circumferences. So we might have to ignore some areas in the bigger circle. Then until another α value is calculated the robo will go according to the previously calculated α value. In that path, there can be objects which were not identified in the initial wall following rounds. (such as objects placed in the middle of the road, which are not connected to any wall).



Then the robot will clash with objects.

Date : 15th of May 2025

We decided the previous method would not be successful.

Then decided to implement left wall following since we have to send the robot through the arena somehow.

We first installed cameras and sensors to the robot.

sensors $\rightarrow$ Two left sensors (left corner sensor and left sensor)

The following code was inserted to the robot.

```
void turn_180(WbDeviceTag left_motor, WbDeviceTag right_motor) {
    // Calculate the duration needed for a 180-degree turn
    double turn_duration_180 = (M_PI * WHEEL_DISTANCE) / (2.0 * turn_speed * WHEEL_RADIUS); // Assuming differential drive kinematics
    // move_robot(left_motor, right_motor, 3.0, 3.0);
    // Execute the turn for the calculated duration
    move_robot_turn(left_motor, right_motor, -turn_speed, turn_speed, turn_duration_180);
}

void turn_right(WbDeviceTag left_motor, WbDeviceTag right_motor) {
    // Calculate the duration needed for a 90-degree turn
    double turn_duration_90 = (M_PI_2 * WHEEL_DISTANCE) / (2.0 * turn_speed * WHEEL_RADIUS); // Assuming differential drive kinematics
    // move_robot(left_motor, right_motor, 3.0, 3.0);
    // Execute the turn for the calculated duration
    move_robot_turn(left_motor, right_motor, turn_speed, -turn_speed, turn_duration_90);
}

void turn_left(WbDeviceTag left_motor, WbDeviceTag right_motor) {
    double turn_duration_90 = (M_PI_2 * WHEEL_DISTANCE) / (2.0 * turn_speed * WHEEL_RADIUS);
    move_robot_turn(left_motor, right_motor, -turn_speed, turn_speed, turn_duration_90);
    // move_robot_turn(left_motor, right_motor, -turn_speed, turn_speed, turn_duration); // Turn left
    // move_robot(left_motor, right_motor, 3.0, 3.0); // Continue after turn
}

void move_forward(WbDeviceTag left_motor, WbDeviceTag right_motor) {
    move_robot(left_motor, right_motor, 3.0, 3.0);
}
```

But this was not very successful because since the path does not have 90° turns and it is circular

```
//leftwall
void left_wall_follow(double max_speed, int front_wall, int left_wall, int left_corner, int left_motor, int right_motor, int left_speed, int right_speed){
    // Decision making
    if (front_wall) {
        printf("Turn right in place\n");
        left_speed = max_speed;
        right_speed = -max_speed;
    } else {
        if (left_wall) {
            printf("Drive forward\n");
            left_speed = max_speed;
            right_speed = max_speed;
        } else {
            printf("Turn right\n");
            left_speed = max_speed / 8;
            right_speed = max_speed;
        }
        if (left_corner) {
            printf("Came too close, drive right\n");
            left_speed = max_speed;
            right_speed = max_speed / 8;
        }
    }
}
```

then

Then we decided to change the algorithm.

```
// Collision avoidance
if (front < 50) {
    // Stop and turn
    left_speed = -2.0;
    right_speed = 2.0;
}
else {
    if (curve_diff > 25) {
        // Turning left curve
        left_speed -= 1.0;
        right_speed += 1.0;
    } else if (curve_diff < -25) {
        // Turning right curve
        left_speed += 1.0;
        right_speed -= 1.0;
    } else {
        // Straight wall - small correction
        left_speed += 0.05 * curve_diff;
        right_speed -= 0.05 * curve_diff;
    }
}
```

we changed
the ~~the~~ algorithm
of wall following

use this concept.
now we can
the robot can
follow the
wall walls
in given task.

Date: 18th of May

Decided that the previous solution would not be suitable. There are not any 90° turns and the path is circular. Therefore it is better if robot can use the sensors and decide an angle itself and turn. So we decided to use PID control.

We started writing the code for the robot using PID

Date: 19th of May

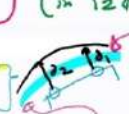
Writing the code was continued on this day too. We researched about PID controlling and try to come up with ways to implement it to our project.

```

1 #include <robot.h>
2 #include <distance_sensor.h>
3 #include <motor.h>
4 #include <math.h>
5 #include <stdio.h>
6
7 #define TIME_STEP 64
8
9 // Constants
10 #define DIST_PROPORTIONAL_CONST 0.834 / 2
11 #define INPUT_DISTANCE 10
12 #define ERROR_DIST 3
13 #define SPEED 3.0 // Adjust motor speed (Robots uses rad/s or % of maxVelocity)
14 #define MAX_SENSOR_VALUE 950
15 #define MAX_OF_SENSOR 800
16 #define MAX_DIFFERENCE 3.00
17 #define MAX_TURN_SPEED 2.0
18 #define CALIBRATION 3.0
19 #define MAX_ALIGN_ANGLE 5.0
20 #define COLLISION_DISTANCE 10.0
21
22 // PID constants
23 float kp_d = 1, kd_d = 0, ki_d = 0;
24 float kp_a = 1, kd_a = 0;
25
26 // State variables
27 float errorD = 0, previousErrorD = 0;
28 float integral = 0, derivative = 0;
29 float outputD = 0, outputA = 0;
30 float previousAngle = 0;
31
32 #ifdef DEVICE_TAG
33 #define leftSensor1, leftSensor2, frontSensor;
34 #define leftMotor, rightMotor;
35
36 float read_sensor(uhDeviceTag sensor) {
37     float value = uh_distance_sensor_get_value(sensor);
38     if (value == 0.0 || value > MAX_SENSOR_VALUE)
39         value = MAX_OF_SENSOR;
40     float distance = value * DIST_PROPORTIONAL_CONST;
41     return distance;
42 }

```

Convert Sensor Value to a distance & use it in check collision (in 120 lms)



$$\text{Current distance} = \frac{d_1 + d_2}{2}$$

give an average Value between the wall & the car

too far

```

43 float current_distance(float read1, float read2) {
44     return (read1 + read2) / 2.0;
45 }

```

```

49 int check_region(float read1, float read2) {
50     float distance = current_distance(read1, read2);
51     if (fabs(distance - INPUT_DISTANCE) > ERROR_DIST) {
52         return (distance > INPUT_DISTANCE) ? -1 : 1;
53     }
54     return 0;
55 }

```

If the distance value

INPUT_DISTANCE; -1 (outside the region)

+1 (Inside the region) too close

If in the region -> 0 (in region)

```

57 void set_motor_speeds(float speedL, float speedR) {
58     uh_motor_set_velocity(leftMotor, speedL);
59     uh_motor_set_velocity(rightMotor, speedR);
60 }
61
62 void reach_distance(float distance1, float distance2) {
63     float currentDistance = current_distance(distance1, distance2);
64     errorD = currentDistance - INPUT_DISTANCE;
65     derivative = errorD - previousErrorD;
66     integral += errorD;
67     outputD = kp_d * errorD + ki_d * integral + kd_d * derivative;
68     previousErrorD = errorD;
69
70     float speedL = SPEED - outputD;
71     float speedR = SPEED + outputD;
72
73     if ((speedL - speedR) > MAX_DIFFERENCE) {
74         speedL = SPEED + MAX_DIFFERENCE;
75         speedR = SPEED - MAX_DIFFERENCE;
76     } else if ((speedL - speedR) < -MAX_DIFFERENCE) {
77         speedL = SPEED - MAX_DIFFERENCE;
78         speedR = SPEED + MAX_DIFFERENCE;
79     }
80     set_motor_speeds(speedL, speedR);
81 }
82
83 void follow_wall(float distance1, float distance2) {

```

Input = 8, 10
Error -> 5, 3

Diverge -> 14 -> 13 -> -1
14 - 13 -> -1
12 -> 4 -> 0
12 - 8 -> 0

3 -> 0 -> +1; but average value doesn't come to be

* black colour Value between 3 & 0
Values are correct. -> It does not detect too close situation
-> doesn't detect obstacles.

```

85 float angle = distance2 - distance1;
86 float derivativeA = angle - previousAngle;
87 outputA = kp_a * angle + kd_a * derivativeA;
88 previousAngle = angle;
89
90 float speedL, speedR;
91
92 if (angle == 0.0) {
93     speedL = SPEED + 1;
94     speedR = SPEED + 1;
95 } else {
96     speedL = SPEED + outputA;
97     speedR = SPEED - outputA;
98 }
99
100 printf("Sensor left (value): %f\n", uh_distance_sensor_get_value(leftSensor1));
101 printf("Sensor right (value): %f\n", uh_distance_sensor_get_value(leftSensor2));
102 printf("Sensor front (value): %f\n", uh_distance_sensor_get_value(frontSensor));
103 printf("Sensor speedL: %f\n", speedL);
104 printf("Sensor speedR: %f\n", speedR);
105 printf("Sensor angle: %f\n", angle);
106 printf("Sensor OutputA: %f\n", outputA);
107
108
109
110
111
112 if ((speedL - speedR) > MAX_TURN_SPEED) {
113     speedL = SPEED + CALIBRATION + MAX_TURN_SPEED;
114     speedR = SPEED - MAX_TURN_SPEED;
115 } else if ((speedL - speedR) < -MAX_TURN_SPEED) {
116     speedL = SPEED + CALIBRATION - MAX_TURN_SPEED;
117     speedR = SPEED + MAX_TURN_SPEED;
118 }
119
120 set_motor_speeds(speedL, speedR);
121 }
122
123 bool check_collision() {
124     float frontDistance = read_sensor(frontSensor);
125     return frontDistance < COLLISION_DISTANCE;
126 }
127
128
129 int main() {

```

Float speedL = SPEED;
Float speedR = SPEED;

here, since we use speed for both L & R speed,

calculation in this part always becomes zero
L - R = 0
-> No point of taking below following.

take the reading of the front sensor & then check whether it is less than that of collision distance. (10)

follow speed of speed 4

```

124 bool check_collision() {
125     float frontDistance = read_sensor(frontSensor);
126     return frontDistance < COLLISION_DISTANCE;
127 }
128
129 int main() {
130     wb_robot_init();
131
132     // Initialize devices
133     leftSensor1 = wb_robot_get_device("left sensor");
134     leftSensor2 = wb_robot_get_device("right sensor");
135     frontSensor = wb_robot_get_device("front sensor");
136
137     wb_distance_sensor_enable(leftSensor1, TIME_STEP);
138     wb_distance_sensor_enable(leftSensor2, TIME_STEP);
139     wb_distance_sensor_enable(frontSensor, TIME_STEP);
140
141     leftMotor = wb_robot_get_device("left wheel motor");
142     rightMotor = wb_robot_get_device("right wheel motor");
143
144     wb_motor_set_position(leftMotor, INFINITY);
145     wb_motor_set_position(rightMotor, INFINITY);
146
147     wb_motor_set_velocity(leftMotor, 0.0);
148     wb_motor_set_velocity(rightMotor, 0.0);
149
150     while (wb_robot_step(TIME_STEP) != -1) {
151         float d1 = read_sensor(leftSensor1);
152         float d2 = read_sensor(leftSensor2);
153         int region = check_region(d1, d2);
154         printf("d1: %f, d2: %f", d1, d2);
155         float align_angle = fabs(d2 - d1);
156
157         //
158         //
159         //
160         if (check_collision()) {
161             //
162             //
163             //
164             //
165             //
166             //
167             //
168             //
169             //
170             //
171             //
172             //
173             //
174             //
175             //
176             //
177             //
178             //
179             //
180             //
181             //
182             //
183             //
184             //
185             //
186             //
187             //
188             //
189             //
190             //
191             //
192             //
193             //
194             //
195             //
196             //
197             //
198             //
199             //
200             //
201             //
202             //
203             //
204             //
205             //
206             //
207             //
208             //
209             //
210             //
211             //
212             //
213             //
214             //
215             //
216             //
217             //
218             //
219             //
220             //
221             //
222             //
223             //
224             //
225             //
226             //
227             //
228             //
229             //
230             //
231             //
232             //
233             //
234             //
235             //
236             //
237             //
238             //
239             //
240             //
241             //
242             //
243             //
244             //
245             //
246             //
247             //
248             //
249             //
250             //
251             //
252             //
253             //
254             //
255             //
256             //
257             //
258             //
259             //
260             //
261             //
262             //
263             //
264             //
265             //
266             //
267             //
268             //
269             //
270             //
271             //
272             //
273             //
274             //
275             //
276             //
277             //
278             //
279             //
280             //
281             //
282             //
283             //
284             //
285             //
286             //
287             //
288             //
289             //
290             //
291             //
292             //
293             //
294             //
295             //
296             //
297             //
298             //
299             //
300             //
301             //
302             //
303             //
304             //
305             //
306             //
307             //
308             //
309             //
310             //
311             //
312             //
313             //
314             //
315             //
316             //
317             //
318             //
319             //
320             //
321             //
322             //
323             //
324             //
325             //
326             //
327             //
328             //
329             //
330             //
331             //
332             //
333             //
334             //
335             //
336             //
337             //
338             //
339             //
340             //
341             //
342             //
343             //
344             //
345             //
346             //
347             //
348             //
349             //
350             //
351             //
352             //
353             //
354             //
355             //
356             //
357             //
358             //
359             //
360             //
361             //
362             //
363             //
364             //
365             //
366             //
367             //
368             //
369             //
370             //
371             //
372             //
373             //
374             //
375             //
376             //
377             //
378             //
379             //
380             //
381             //
382             //
383             //
384             //
385             //
386             //
387             //
388             //
389             //
390             //
391             //
392             //
393             //
394             //
395             //
396             //
397             //
398             //
399             //
400             //
401             //
402             //
403             //
404             //
405             //
406             //
407             //
408             //
409             //
410             //
411             //
412             //
413             //
414             //
415             //
416             //
417             //
418             //
419             //
420             //
421             //
422             //
423             //
424             //
425             //
426             //
427             //
428             //
429             //
430             //
431             //
432             //
433             //
434             //
435             //
436             //
437             //
438             //
439             //
440             //
441             //
442             //
443             //
444             //
445             //
446             //
447             //
448             //
449             //
450             //
451             //
452             //
453             //
454             //
455             //
456             //
457             //
458             //
459             //
460             //
461             //
462             //
463             //
464             //
465             //
466             //
467             //
468             //
469             //
470             //
471             //
472             //
473             //
474             //
475             //
476             //
477             //
478             //
479             //
480             //
481             //
482             //
483             //
484             //
485             //
486             //
487             //
488             //
489             //
490             //
491             //
492             //
493             //
494             //
495             //
496             //
497             //
498             //
499             //
500             //
501             //
502             //
503             //
504             //
505             //
506             //
507             //
508             //
509             //
510             //
511             //
512             //
513             //
514             //
515             //
516             //
517             //
518             //
519             //
520             //
521             //
522             //
523             //
524             //
525             //
526             //
527             //
528             //
529             //
530             //
531             //
532             //
533             //
534             //
535             //
536             //
537             //
538             //
539             //
540             //
541             //
542             //
543             //
544             //
545             //
546             //
547             //
548             //
549             //
550             //
551             //
552             //
553             //
554             //
555             //
556             //
557             //
558             //
559             //
560             //
561             //
562             //
563             //
564             //
565             //
566             //
567             //
568             //
569             //
570             //
571             //
572             //
573             //
574             //
575             //
576             //
577             //
578             //
579             //
580             //
581             //
582             //
583             //
584             //
585             //
586             //
587             //
588             //
589             //
590             //
591             //
592             //
593             //
594             //
595             //
596             //
597             //
598             //
599             //
600             //
601             //
602             //
603             //
604             //
605             //
606             //
607             //
608             //
609             //
610             //
611             //
612             //
613             //
614             //
615             //
616             //
617             //
618             //
619             //
620             //
621             //
622             //
623             //
624             //
625             //
626             //
627             //
628             //
629             //
630             //
631             //
632             //
633             //
634             //
635             //
636             //
637             //
638             //
639             //
640             //
641             //
642             //
643             //
644             //
645             //
646             //
647             //
648             //
649             //
650             //
651             //
652             //
653             //
654             //
655             //
656             //
657             //
658             //
659             //
660             //
661             //
662             //
663             //
664             //
665             //
666             //
667             //
668             //
669             //
670             //
671             //
672             //
673             //
674             //
675             //
676             //
677             //
678             //
679             //
680             //
681             //
682             //
683             //
684             //
685             //
686             //
687             //
688             //
689             //
690             //
691             //
692             //
693             //
694             //
695             //
696             //
697             //
698             //
699             //
700             //
701             //
702             //
703             //
704             //
705             //
706             //
707             //
708             //
709             //
710             //
711             //
712             //
713             //
714             //
715             //
716             //
717             //
718             //
719             //
720             //
721             //
722             //
723             //
724             //
725             //
726             //
727             //
728             //
729             //
730             //
731             //
732             //
733             //
734             //
735             //
736             //
737             //
738             //
739             //
740             //
741             //
742             //
743             //
744             //
745             //
746             //
747             //
748             //
749             //
750             //
751             //
752             //
753             //
754             //
755             //
756             //
757             //
758             //
759             //
760             //
761             //
762             //
763             //
764             //
765             //
766             //
767             //
768             //
769             //
770             //
771             //
772             //
773             //
774             //
775             //
776             //
777             //
778             //
779             //
780             //
781             //
782             //
783             //
784             //
785             //
786             //
787             //
788             //
789             //
790             //
791             //
792             //
793             //
794             //
795             //
796             //
797             //
798             //
799             //
800             //
801             //
802             //
803             //
804             //
805             //
806             //
807             //
808             //
809             //
810             //
811             //
812             //
813             //
814             //
815             //
816             //
817             //
818             //
819             //
820             //
821             //
822             //
823             //
824             //
825             //
826             //
827             //
828             //
829             //
830             //
831             //
832             //
833             //
834             //
835             //
836             //
837             //
838             //
839             //
840             //
841             //
842             //
843             //
844             //
845             //
846             //
847             //
848             //
849             //
850             //
851             //
852             //
853             //
854             //
855             //
856             //
857             //
858             //
859             //
860             //
861             //
862             //
863             //
864             //
865             //
866             //
867             //
868             //
869             //
870             //
871             //
872             //
873             //
874             //
875             //
876             //
877             //
878             //
879             //
880             //
881             //
882             //
883             //
884             //
885             //
886             //
887             //
888             //
889             //
890             //
891             //
892             //
893             //
894             //
895             //
896             //
897             //
898             //
899             //
900             //
901             //
902             //
903             //
904             //
905             //
906             //
907             //
908             //
909             //
910             //
911             //
912             //
913             //
914             //
915             //
916             //
917             //
918             //
919             //
920             //
921             //
922             //
923             //
924             //
925             //
926             //
927             //
928             //
929             //
930             //
931             //
932             //
933             //
934             //
935             //
936             //
937             //
938             //
939             //
940             //
941             //
942             //
943             //
944             //
945             //
946             //
947             //
948             //
949             //
950             //
951             //
952             //
953             //
954             //
955             //
956             //
957             //
958             //
959             //
960             //
961             //
962             //
963             //
964             //
965             //
966             //
967             //
968             //
969             //
970             //
971             //
972             //
973             //
974             //
975             //
976             //
977             //
978             //
979             //
980             //
981             //
982             //
983             //
984             //
985             //
986             //
987             //
988             //
989             //
990             //
991             //
992             //
993             //
994             //
995             //
996             //
997             //
998             //
999             //
1000            //

```

take the reading of front sensor & then check whether it is less than that of collision distance. (10)

01:30:688 ← without line following speed 4
 01:40 ← with line following speed 4
 01:39 ← wall following speed 3
 01:41 ← wall following speed 3

we should change the right direction to turn

here even you remove follow wall part, still the robot is working

above pictures shows the algorithm we used. For PID controlling.

Here the main control idea was, the robot use distance sensors (left, right, front) to perform wall following, obstacle detection & collision avoidance. The logic is mainly structured around:

1. wall following Behaviour.

- robot calculates an average distance from side sensors
- then this average is compared with INPUT-DISTANCE to decide
 - 1 → Too far from the wall
 - 0 → In the correct region.
 - +1 → Too close to wall.

Then PID controller adjust the motor speeds & keep the robot at a steady distance from the wall.

We use below concept to collision Detection.

```
if (check_collision())  
    set_motor_speeds(SPEED + MAX_DIFFERENCE, SPEED - MAX_DIFFERENCE);
```

∴ as a conclusion of the day, the robot uses sensor based PID control for wall following, collision detection using the front sensor, & alignment checks to navigate its environment smoothly & safely. All decisions are based on dynamic sensor readings.

Date: 20th of May 2025

After writing the code, we inserted it to the robot, Next challenge was calibrating the code. K_p , K_i , K_d values were found respectively. We changed values one by one until our robot followed the path very smoothly. This was a huge achievement for us.

Date : 21st of May 2025

After calibrating the k_p , k_i , k_d , k_a values we realized that even a minute change in them can drastically change the path of the robot. And in the competition though we get a sample track at the beginning before the competition, calibrating the robot according to that path might not work on the real path. Since we do not get to calibrate the robot according to the competition track, we thought it would be better to use auto calibration instead of this manual calibrating method.

So we then started learning about auto calibration codes, and finally inserted it to our code in a suitable way. Then when the world was run, it went smoothly

Concept:

In the webot simulation, we implemented a PID control system to regulate the robot's movement effectively by continuously monitoring error logs, and detecting zero crossings. We calculated oscillation period (T_u) & derivative (T_d) constants based on established tuning formulas. The system ensures stability & precision by dynamically adjusting PID parameters, allowing robot to navigate efficiently while minimizing errors.

```

1 import numpy as np
2 import re
3 import os
4
5 ERROR_LOG = "error.txt"
6 PID_LOG = "pid_constants.txt"
7 KU = 0.5 # Ultimate gain (manual or auto)
8
9
10 import subprocess
11 import os
12 import time
13 count=0
14
15 def launch_webots():
16     cmd = [
17         r"C:\Program Files\Webots\mvs64\mingw64\bin\webots.exe",
18         "--stdout",
19         "D:\Maze Solver\New folder\Parabots Resources\worlds\para bots.wbt"
20     ]
21     # Redirect Webots output to DEVNULL (no terminal clutter)
22     subprocess.Popen(cmd, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
23
24 def read_error_log():
25     times, errors = [], []
26     try:
27         with open(ERROR_LOG, "r") as f:
28             for line in f:
29                 # print(f"[DEBUG] Line: {line.strip()}")
30                 match = re.match(r"Time: ([\d.]+), Error: ([\d.]+)", line)

```

```

31         if match:
32             t = float(match.group(1))
33             e = float(match.group(2))
34             print(f"[DEBUG] Matched - Time: {t}, Error: {e}")
35             times.append(t)
36             errors.append(e)
37         else:
38             print(f"[DEBUG] No match")
39     except FileNotFoundError:
40         return [], []
41
42 return times, errors
43
44 def detect_zero_crossings(times, errors):
45     zero_crossings = []
46     for i in range(1, len(errors)):
47         if errors[i-1] * errors[i] < 0:
48             t1, t2 = times[i-1], times[i]
49             e1, e2 = errors[i-1], errors[i]
50             crossing_time = t1 + (0 - e1) * (t2 - t1) / (e2 - e1)
51             zero_crossings.append(crossing_time)
52     return zero_crossings
53
54 def calculate_tu(zero_crossings):
55     if len(zero_crossings) < 3:
56         return None
57     periods = [zero_crossings[i+2] - zero_crossings[i] for i in range(len(zero_crossings) - 2)]

```

```

58     return np.mean(periods)
59
60 def calculate_pid_constants(ku, tu):
61     kp = 0.6 * ku
62     ki = 1.2 * ku / tu
63     kd = 0.075 * ku * tu
64     return kp, ki, kd
65
66 def save_pid_constants(kp, ki, kd):
67     with open(PID_LOG, 'w') as f:
68         f.write(f"Kp: {kp:.4f}\nKi: {ki:.4f}\nKd: {kd:.4f}\n")
69     print(f"[Python] Updated PID: Kp={kp:.4f}, Ki={ki:.4f}, Kd={kd:.4f}")
70
71 def monitor_and_update():
72     count = 0
73     while count < 100:
74         time.sleep(5)
75         times, errors = read_error_log()
76         print("!!!")
77         if len(times) < 5:
78             continue
79         zero_crossings = detect_zero_crossings(times, errors)
80         tu = calculate_tu(zero_crossings)
81         if tu:
82             kp, ki, kd = calculate_pid_constants(ku, tu)
83             save_pid_constants(kp, ki, kd)
84         else:
85             print(f"[!] Not enough zero crossings yet.")
86         count+=1
87     print(times, zero_crossings)

```

```

88
89 # ----- MAIN -----
90 if __name__ == "__main__":
91     if os.path.exists(PID_LOG):
92         os.remove(PID_LOG)
93     if os.path.exists(ERROR_LOG):
94         os.remove(ERROR_LOG)
95
96     print("[*] Starting Webots...")
97     launch_webots()
98
99     print("[*] Monitoring and auto-tuning PID...")
100    monitor_and_update()
101    print("!!!")
102
103
104

```

periods. → The time between two identical error points.

What it does:
This test computer calculates full oscillation periods by measuring the time between every pair of zero-crossings that are two indices apart.
In simple terms:
• $zero_crossings[0]$ = start of an oscillation
• $zero_crossings[0+2]$ = start of the next full oscillation
• $zero_crossings[0+2] - zero_crossings[0]$ = one full period (T)

Math Examples:

Let's say we have a list of zero crossings (in seconds):

zero_crossings = [1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0]

These represent the times when the signal crosses zero (up and down alternating).

Find full periods using:

periods = [zero_crossings[i+2] - zero_crossings[i] for i in range(len(zero_crossings) - 2)]

Let's compute:

1. $2.0 - 1.0 = 1.0$

2. $3.0 - 1.5 = 1.5$

3. $3.5 - 2.0 = 1.5$

4. $4.0 - 2.5 = 1.5$

So we get:

periods = [1.0, 1.5, 1.5, 1.5]

zero crossings = [1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0]
periods = [2.0 - 1.0]
[3.0 - 1.5]
[3.5 - 2.0]
[4.0 - 2.5]

$e(t) = e_1 + (e_2 - e_1) \cdot \frac{(t - t_1)}{(t_2 - t_1)}$
error in zero crossing
error distance
error distance

When $e(t) = 0$ error in zero crossing time is
 $t = t_1 + (0 - e_1) \cdot \frac{(t_2 - t_1)}{(t_2 - t_1)}$

if $x > 0$
if x ;

Date: 22nd of May 2025

Next challenge was that, in the competition we get 5 rounds and during those 5 rounds our robot should optimize the path. But even in auto calibration PID control code that does not happen. Even though, the robot follows paths very smoothly, the time it takes to complete the task does not decrease significantly. So we brainstormed ways to do that. Later we identified that machine learning is one such way of doing it. Then we thought to try and use it. Nobody in the group knew what it is and how it is done. So, we started learning about it.

Date: 23rd of May 2025

Today we spent searching what is machine learning, how it is done, what is the logic and stuff. But we were not very successful, no one could understand fully how it is done.

Date = 24th of May 2025

Continued learning machine learning and tried to recreate a previous project to get an experience. But we were not successful. Today was also spent learning about machine learning.

Date : 25th of May 2025

Today we could come up with a code after looking at multiple examples. It is MRL model

First it runs the path by using train_rl file it gives rewards based on the action that it takes during the training / observation session.

If it gives a correct action

eg: if the robot is near the object skip it.

It will generate a high reward. (+)

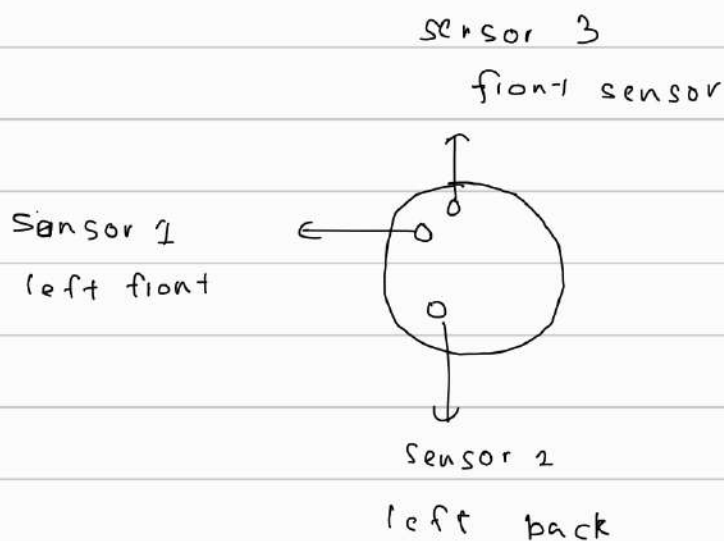
Finally after many runtimes it makes a file called ppo-wall-follower which contains the trained neural model. It runs through the run-trained file.

Finally the robot doesn't show efficient Path optimization. It is due to that the path is almost clear and the model cannot learn much from the path. So, the PID works well than that of RL model. So, we decided to develop PID for auto tuning in each instances for path optimization. It may reduce the time to complete the lap by lap, by recording each error value and processing it to give efficient k values using Ziegler - Nicholas Algorithm

Date : 26th of May 2025

Incorporating machine learning was not very successful due to our limited knowledge. Plus machine learning is complicated and it might not be necessary to complete the task.

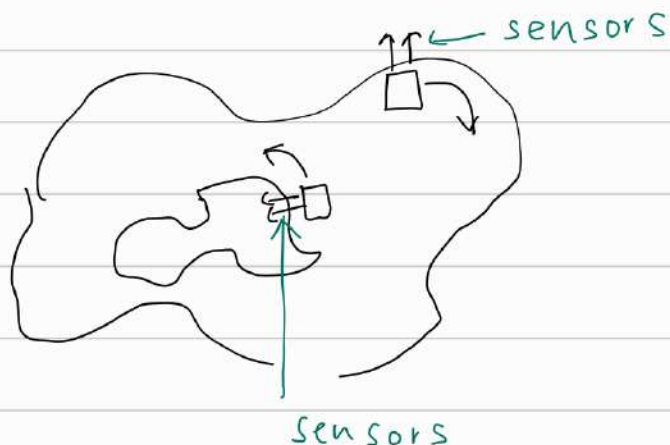
Therefore,
decided to move forward with the auto calibration code using a left wall following method. In here we placed two sensors on the left side of the robot. And another at the front of the robot.



At first we decided to send the robot following left wall in the first round and following the right wall in the second round and third round onwards use whichever the path that took less time than the other path.

Date : 27th of May 2025

Today was spent trying the right wall following.
Same robot following the other wall

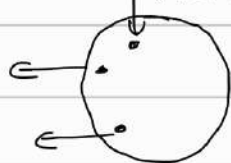


It was successful. At the Q&A session, it was said that using epuck would reduce our marks.

Therefore, we started building a robot from the scratch. We decided to make a cylindrical robot because then it would be easier to take bends. Sensors were placed as the following image of robot was made according to the given dimensions.

(If) Front sensor

left front sensor



(fs)

left back sensor

(lb)

Date : 28th of May 2025

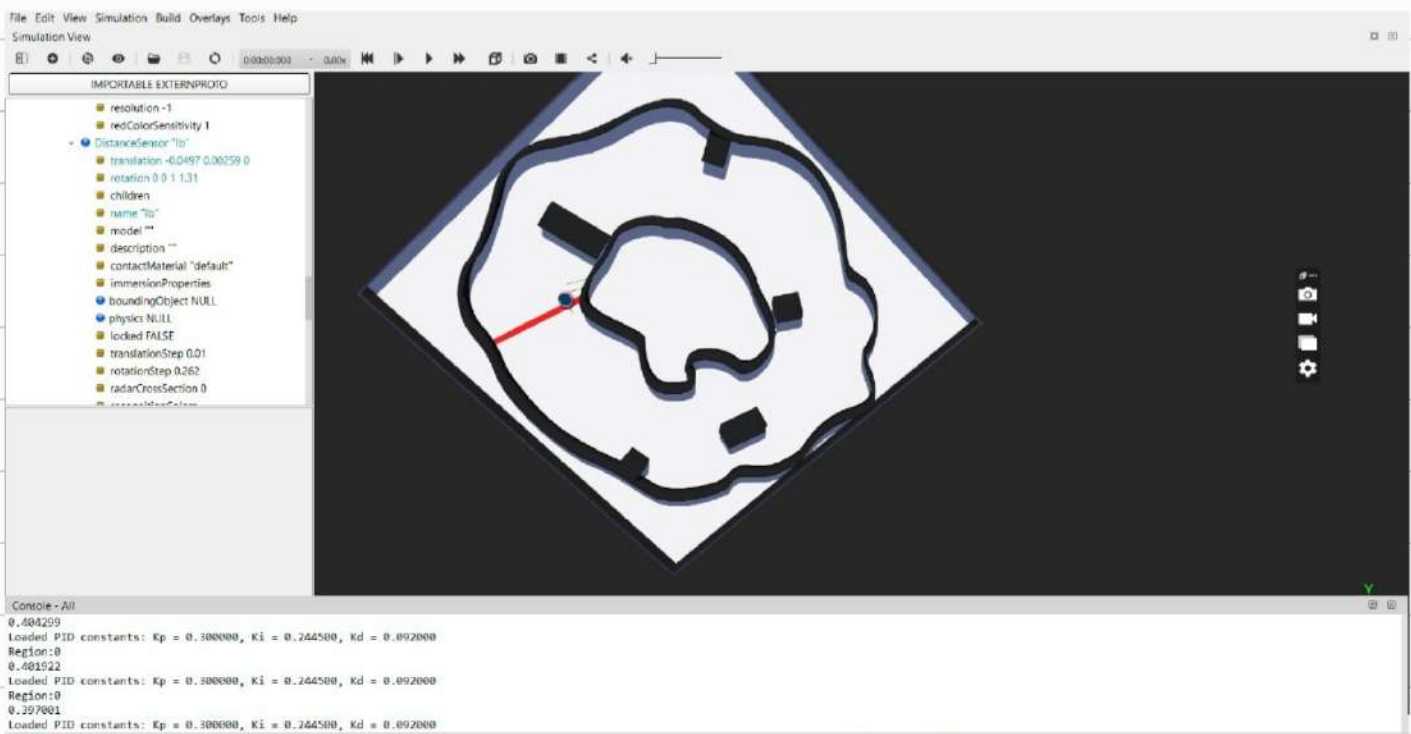
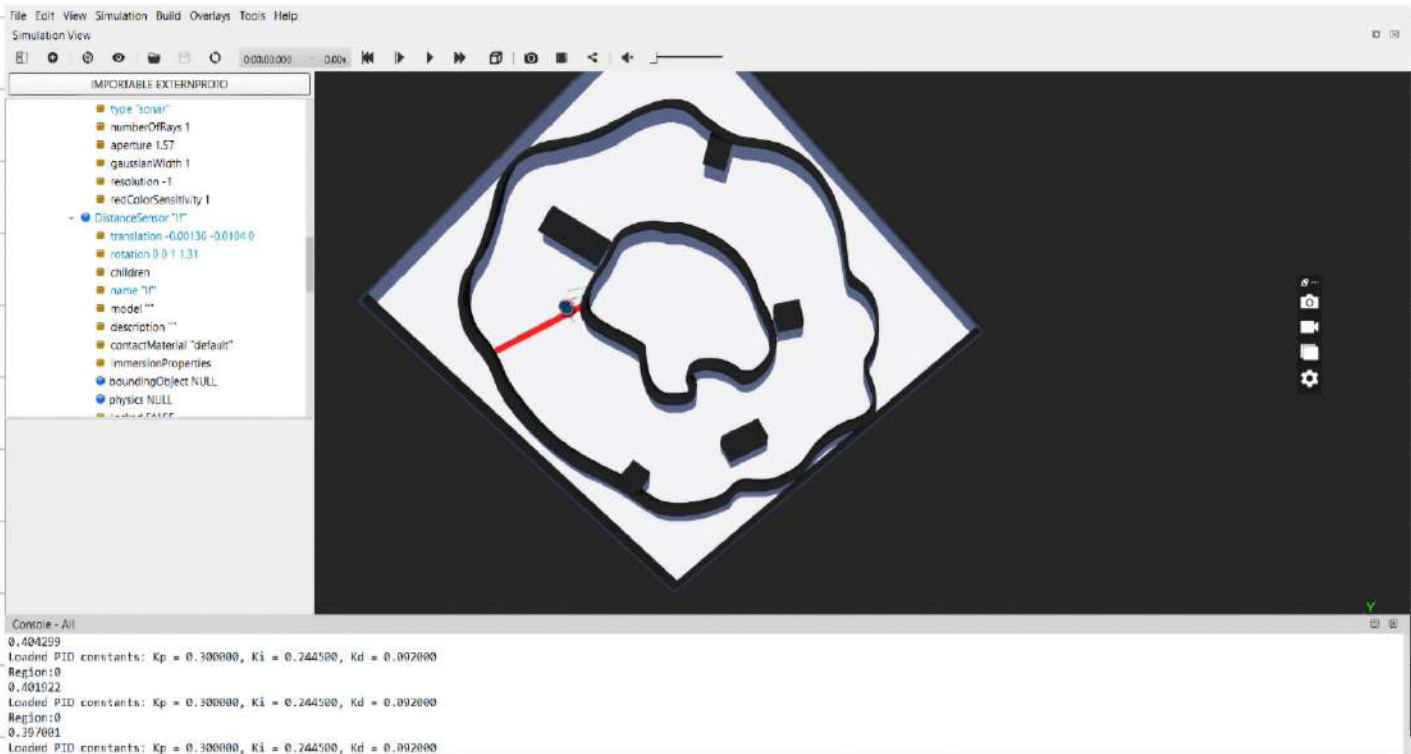
Today we optimized the code. When sensors are not sensing anything, the robot will continue to go forward.

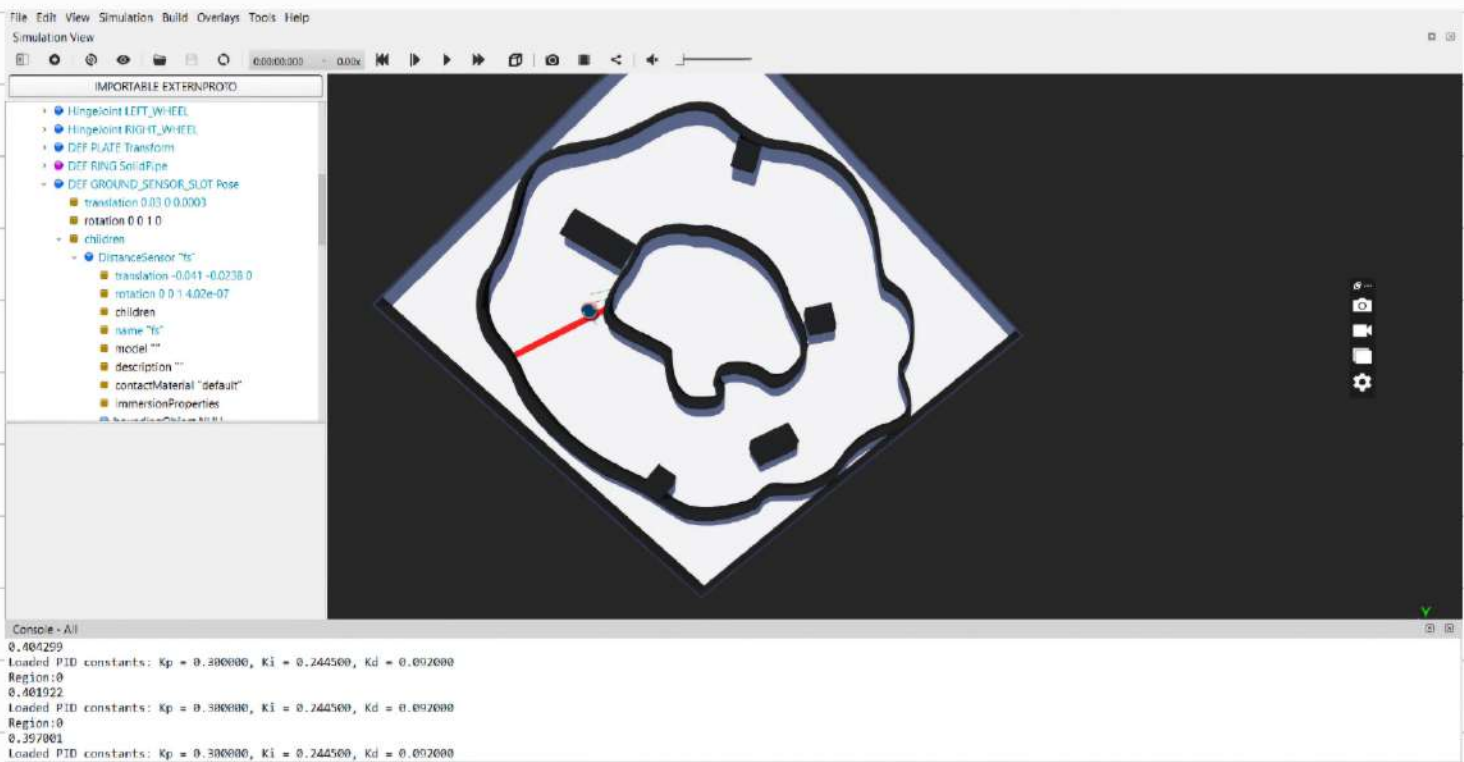
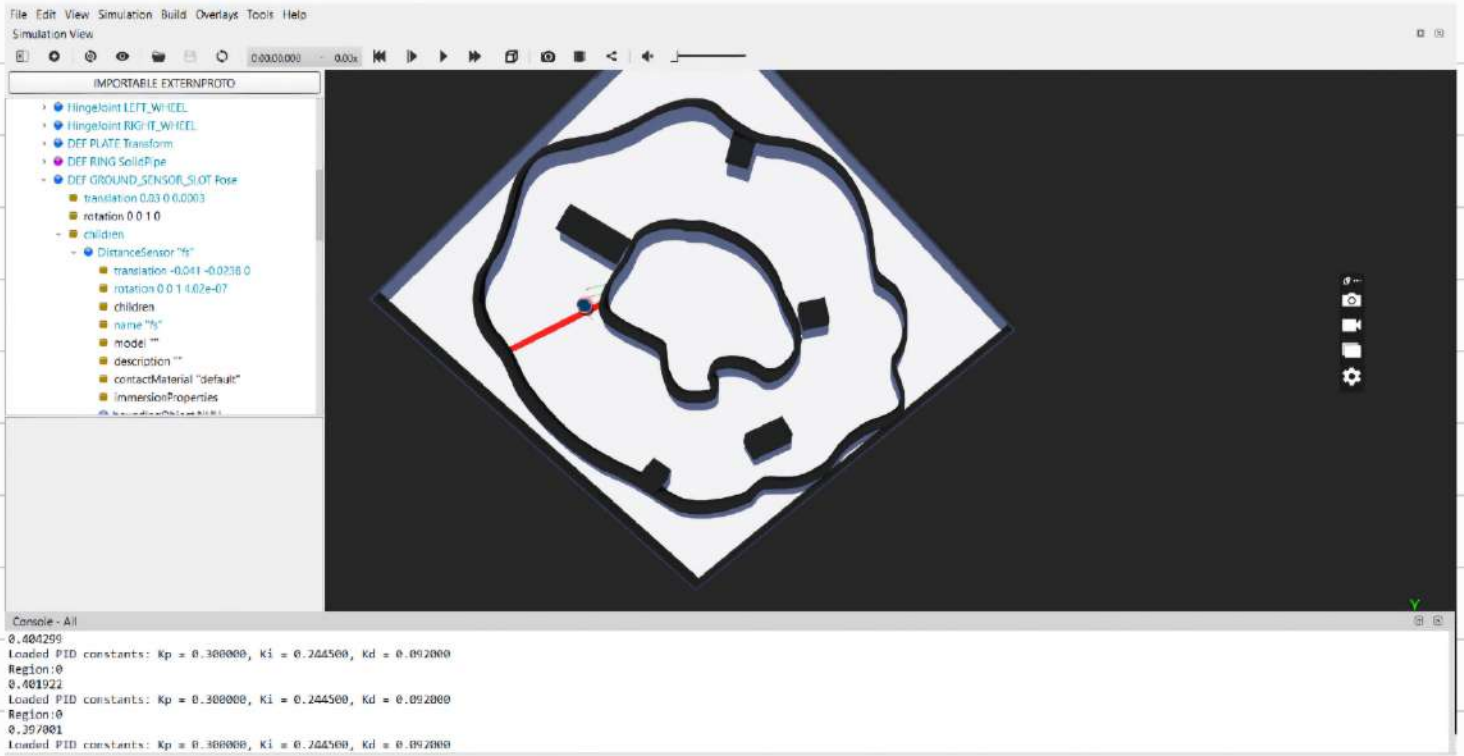
The robot uses four distance sensors (Lf, lb, fs & lc) to monitor its surroundings. The two side sensors (Lf and lb) are used to calculate the robot's average distance from the wall. A PID controller is applied to minimize the deviation from a desired target distance (INPUT_DISTANCE), adjusting the left and right motor speeds to keep the robot aligned with the wall.

The PID constants (k_p , k_i , k_d) are loaded dynamically from an external file (pid_constants.txt), allowing real-time tuning without modifying the code. The values influence how the robot corrects its path in response to distance errors. By continuously adjusting motor speeds based on PID output, the robot optimizes its path to maintain a smooth and stable trajectory along the wall.

A front sensor detects obstacles and triggers a turning maneuver to avoid collisions. Additionally, sensor readings and error values are logged to a file (error.txt) for analysis and performance evaluation.

This combination of sensor feedback, PID control and dynamic parameter tuning helps the robot to optimize its path and maintain efficient wall-following behaviour under varying conditions. At first we tuned the base speed to 3, but later turned it to 6. (The maximum in webots)





Date : 29th of May 2025

Today we started to make the submission video. It has 5 rounds of the robot following the left wall and another 5 rounds of the robot following the right wall.

#Outer Wall Performance

Lap number	Start time	End time	Duration
1	00:50	11:04	10:14
2	11:04	15:24	04:20
3	15:24	19:28	04:04
4	19:28	23:34	04:06
5	23:34	27:38	04:04

#Inner Wall Performance

Lap number	Start time	End time	Duration
1	00:09	07:34	07:25
2	07:34	11:24	03:50
3	11:24	15:08	03:44
4	15:08	19:04	03:56
5	19:04	22:50	03:46